# Assessing the Impact of Starting Card Arrangements on Klondike Solitaire Solvability

Joshua Stoneback

A Senior Thesis submitted in partial fulfillment
of the requirements for graduation
in the Honors Program
Liberty University
Spring 2025

Acceptance of Senior Honors Thesis

This Senior Honors Thesis is accepted in partial
fulfillment of the requirements for graduation from the
Honors Program of Liberty University.

_____
Daniel O'Malley, Ph.D.
Thesis Chair

_____
Tim Van Voorhis, Ph.D.
Committee Member

_____
Emily C. Knowles, D.B.A.
Assistant Honors Director

_____
Date

**Abstract**

Klondike solitaire gained widespread recognition after its inclusion in Microsoft Windows in 1990. Questions regarding the probability of winning an arbitrary game of solitaire have intrigued hobbyists and researchers for decades. This thesis examines an extension to this question, aimed at calculating the solvability impact of the initial, visible cards in the tableau when a player starts a game. The solitaire solver Solvitaire is used, in conjunction with multiple custom JSON generators, to determine the solvability of multiple types of visible tableau configurations. This study demonstrates that the solvability chances of a game with four aces or kings visible is considerably higher than other homogeneous configurations. The methodology employed in this study can inform solitaire solvers for years to come.

## Assessing the Impact of Starting Card Arrangements on Klondike Solitaire Solvability

Solitaire games, known as patience games outside the United States, have been popular for hundreds of years (Ross & Healey, 1963). These games often involve the arrangement of playing cards into a systematic order, which can vary greatly depending on the type of game played. Perhaps the most popular version of solitaire today is Klondike solitaire. The popularity of this version was fueled by the inclusion of Solitaire as an application in the Windows operating system. The game was introduced as *Windows Solitaire* in 1990, included as part of Windows 3.0 (Jensen, 2020). The game was included to help users learn how to drag and drop items on the screen with a mouse but quickly became the most-used application on Windows, even causing productivity concerns in offices where it was installed (Garreau, 1994; Jensen, 2020). Klondike solitaire has maintained this popularity in the 21st century, with the Microsoft implementation of the game alone being played over 100 million times per day in 2020 (Jensen, 2020).

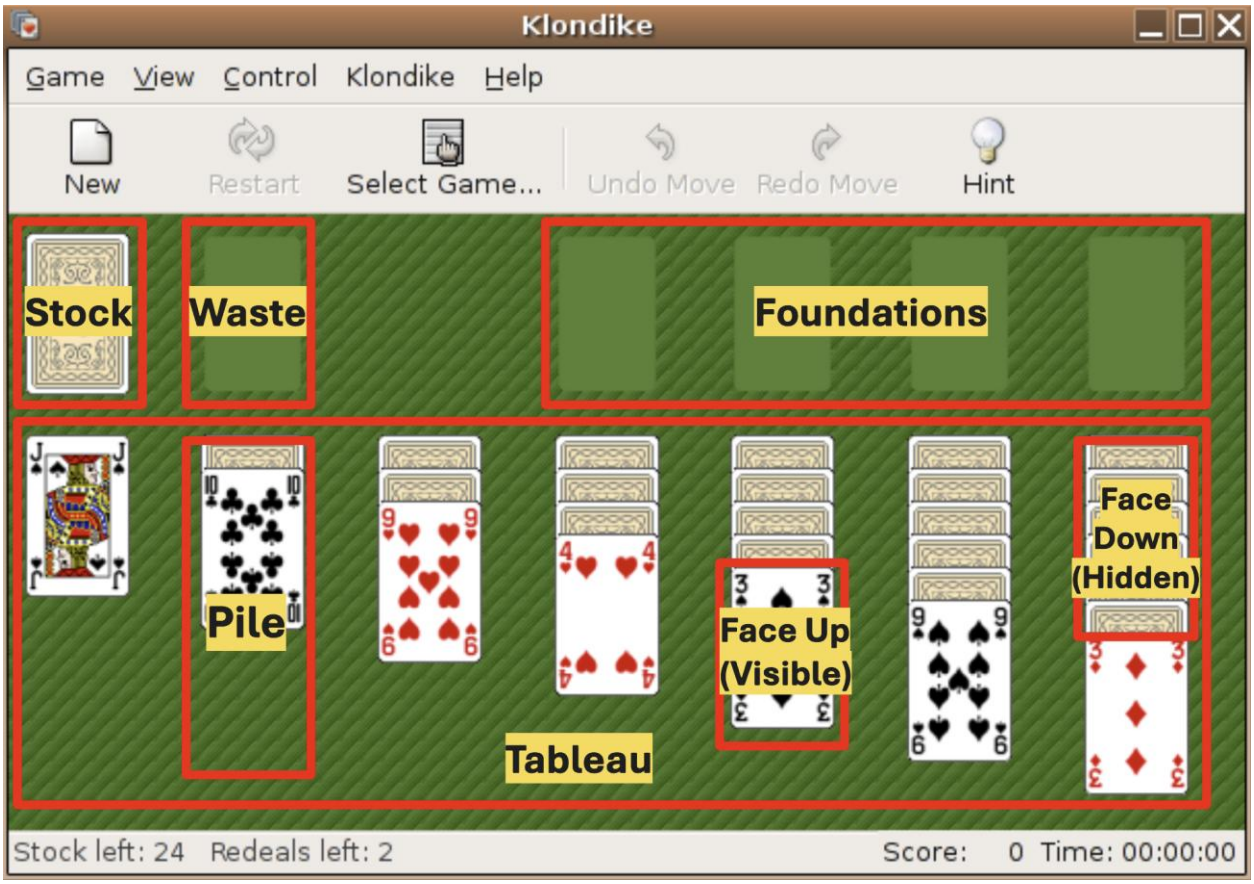
While enjoyers of solitaire have been interested in the probability of winning a game for over a century (Cavendish, 1890), only in recent years have computational efforts been taken to calculate a winnability percentage. These efforts have culminated in the derivation of an upper bound for solvability to 81.945% ± 0.084% (Blake & Gent, 2024). Blake and Gent left their program used for calculating these solvability percentages open source, encouraging others to continue their research. One extension to this research, considering the solvability of games based on characteristics of the initial visible tableau, is considered in this study.

## Terminology and Rules for Klondike Solitaire

Before solvability analysis of Klondike solitaire can occur, it is important to clarify key terminology surrounding the game and the specific ruleset that has been chosen for this analysis.

**Figure 1**

*Example configuration for a game of Klondike solitaire (with annotations)*




*Note*. The GNOME Project. (2006) [Klondike (solitaire)]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Klondike_(solitare).png

Although there are many versions of solitaire, henceforth in this paper, any mention of solitaire refers to Klondike solitaire unless otherwise stated. A possible starting position of a game of solitaire is shown in Figure 1. The objective of the game is to move all 52 cards to the foundations, with one pile for each of the four suits. Each pile is ordered with ace at the bottom, followed by 2 through 10, jack, queen, and king; cards of each suit must be moved to the foundation strictly in that order. To complete this task, it is likely that cards in the tableau will

need to be rearranged. Visible cards on the tableau may be stacked with other visible cards if their ranks are consecutive and decreasing (e.g., a 9 can be stacked below a 10). Additionally, cards in the tableau must be stacked with alternating colors. Cards from the stock are flipped to the waste pile 3 at a time, with the top card considered playable to either the foundations or the tableau. Once this pile runs out, it is reset in the same order. When one of the 7 piles in the tableau becomes empty, a king can be placed in the empty spot (if it is visible/playable). While variants of solitaire exist where a different number of cards from the stock are flipped at once or number of passes through the deck is limited, the described configuration is consistent with prior solvability research (Bjarnason et al., 2009; Blake & Gent, 2024; Yan et al., 2004) and is used for the entirety of this study.

## History of Solving Klondike Solitaire

The probability of solving a game of solitaire has been of great interest to hobbyists for years, with many books on solitaire speculating on the chances of success in a game. One of the earliest examples in literature comes from the late 1940s, where Klondike solitaire is said to be won in 1 in 30 games (Morehead & Mott-Smith, 1949). A few decades later, Parlett (1980) described the game as “odds against” in his comprehensive review of solitaire games.

While these solvability estimates were likely based on personal experiences more than computational analysis, some researchers took a deeper interest in whether an analytical solution exists. In 1946, Stanislaw Ulam, while recovering from an illness, contemplated the chances of a solitaire game being successful (Eckhardt, 1987). After spending some time considering a combinatorial solution, he considered that a more practical solution would be to play the game many times and count the number of successful games. He envisioned this and similar tasks feasible with the new era of fast computers. This led to the development of Monte Carlo

methods, which were used by Ulam to solve problems of neutron diffusion and by modern solitaire researchers to answer the question Ulam had contemplated initially (Bjarnason et al., 2009; Blake & Gent, 2024; Yan et al., 2004).

## Modern Research

Monte Carlo methods have been utilized in all serious solvability research efforts due in part to the extremely large state space of solitaire's starting configurations (Bjarnason et al., 2009; Blake & Gent, 2024; Yan et al., 2004). Since each of the 52 cards can be placed in any order, there are 52! possible starting configurations, a prohibitively large number for any kind of comprehensive analysis. Similar issues arise with perfect information games that have a high number of possible moves, such as chess (Shannon, 1950). Unlike games such as chess, solitaire is a single-player game that can be easily programmed for a computer to solve, though it might be computationally expensive. Thus, modern methods utilize Monte Carlo methods to simulate a set number of games and use those results to obtain confidence intervals (Bjarnason et al., 2009; Blake & Gent, 2024; Yan et al., 2004).

### *Thoughtful Solitaire*

Yan et al. (2004) shifted academic attention towards solitaire solvability when they proclaimed that it is "one of the embarrassments of applied mathematics" that the solvability probability of solitaire is not known. Yan et al. introduced a game called thoughtful solitaire, which is identical to regular solitaire with the key exception that the identity of all cards are known. It is important to note that this is not simply playing solitaire with all the cards face up, playing in that manner could change the possible moves in the tableau. Thoughtful solitaire creates a perfect information version of solitaire that can be easier solved by an algorithm, as opposed to the partially observable system of standard solitaire.

Yan et al. (2004) learned of thoughtful solitaire from a senior American mathematician, who considered the game to be more thought-provoking and challenging than standard solitaire. This mathematician carefully played 2000 games and achieved a win rate of 36.6%, which was used as a benchmark for a thoughtful solitaire computational solver. Their best results come from the use of a rollout method with 5 rollouts, where they were able to achieve a 99% confidence interval of [61.86%, 78.54%]. This wide confidence interval is a result of a low number of samples (n = 200), caused in part by a very long average run-time per game (1 hour 45 minutes).

### *Interpretation of Confidence Intervals to Standard Solitaire*

Many of the confidence intervals in this analysis are for thoughtful solitaire, not standard Klondike solitaire. Any game that is not solvable in thoughtful solitaire is necessarily also not solvable in solitaire, as the two games have identical movement rules. This does not mean that all solvable games in thoughtful solitaire are solvable in standard solitaire, however. A best strategy may lead the player to a position where a game is no longer winnable without undoing moves. As such, any solvability probability for thoughtful solitaire should be treated as an upper bound for the solvability probability of standard solitaire (Blake & Gent, 2024).

### *Further Upper and Lower Bounding of Solitaire*

A few years after solitaire was declared an embarrassment of applied mathematics (Yan et al., 2004), Bjarnason et al. (2007) greatly improved on previous results for thoughtful solitaire, achieving a 99% confidence interval of [82%, 91.44%]. They were able to achieve this result through using a multistage nested rollout algorithm with 3 levels of nested rollout search for H1 and a single rollout search for H2. These results were also faster than previous results, with an average solution time of 32 minutes per game. This paper was also significant for the introduction of a $K^+$ tree. Since there is no limit to the number of times a player may turn cards

from the stock, all cards that are reachable with any number of terms are considered available for play. This decreases the number of meaningless moves a solver may take and eliminates the option to draw from the stock, since all cards are always available. If a card from the stock is played, the available cards update accordingly.

Unsatisfied with simply an improved upper bound for the solvability of the standard solitaire game, Bjarnason et al. (2009) attempted to lower-bound solitaire by introducing a probabilistic planning algorithm as a solver for the partially observable game of standard solitaire. His best results come from a combination of sparse sampling and UCT, a Monte Carlo method that has proven effective in the complex game of Go (Gelly & Silver, 2007). They were able to achieve a 99% confidence interval of [33.04%, 38.20%] for solving standard solitaire, with an average run-time of about 25 minutes per game. Thus, a worst-case confidence interval for the solvability of standard solitaire was determined to be [33.04%, 91.44%].

**Solvitaire**

Blake and Gent (2024) achieved a breakthrough in calculating the solvability of thoughtful solitaire with their development of Solvitaire, a general solver that can determine solvability percentages for a wide variety of single-player card games. They were able to achieve a solvability 95% confidence interval of [81.861%, 82.029%], which represents a 30-fold reduction in confidence interval over the best previous result. Blake and Gent developed Solvitaire using ideas from a combination of academic research (Bjarnason et al., 2007) and non-academic research (Birrell, 2017; Wolter, 2013). Despite research from both academic and non-academic sources, the best winnability percentages came from non-academic sources, prior to Solvitaire (Blake & Gent, 2024). In the case of Klondike solitaire, the best previous result was [81.177%, 87.173%], a significant improvement on previous academic results (Birrell, 2017;

Blake & Gent, 2024). Solvitaire represents the first general solitaire solver that performs highly on many different single-player card games, including Klondike solitaire (Blake & Gent, 2024).

### *Solvitaire Implementation*

Solvitaire is a depth-first backtracking search solver that operates over the state space of legal card configurations (Blake & Gent, 2024). It employs several Artificial Intelligence techniques that have proven effective in previous solvers (Birrell, 2017; Wolter, 2013) and combines them into a single, general solver (Blake & Gent, 2024).

**Exhaustive Depth-First Search.** A key characteristic of Solvitaire is its optimization as an exhaustive search for unwinnable games, that is, a game is not declared to be unwinnable until all possible positions that come from the starting position are considered (Blake & Gent, 2024). This ensures that any game that is declared to be unwinnable is also proved to be unwinnable by exhausting all possible positions. Additionally, the K+ representation of stock is used (Bjarnason et al., 2007; Blake & Gent, 2024). While this increases the branching rate, it also greatly reduces search depth and ensures that moves are not made to pointlessly draw cards from the stock.

**Transposition Tables.** Transposition tables (Greenblatt et al., 1967; Smith, 2005) are used in Solvitaire to avoid considering the same position twice within a single game. While this can lead to suboptimal solutions, this is acceptable as the primary objective is to find any solution, not the most optimal solution (Akagi et al., 2010; Blake & Gent, 2024). If the transposition table becomes full, elements are removed on a least-recently used basis (Blake & Gent, 2024). The exception to that rule is that elements that are ancestors of the current state always remain in the transposition table to avoid the occurrence of loops.

**Symmetry.** Symmetry in solitaire occurs when nonidentical positions are equivalent and can lead to redundant search if not addressed (Blake & Gent, 2024; Gent et al., 2006). Common

examples of these positions include situations where piles in the tableau are identical but in different spaces, and suits are switched in a way that does not affect the red-black build policy of the game. This is addressed by expressing each state in a canonical form within the transposition table, only expressing necessary information to distinguish between each state (Blake & Gent, 2024). For example, a king can be moved to either of two empty piles, there is no difference between the two. With symmetry, a move that shifts a built pile led by a king from one open pile to another will not be considered.

**Dominances.** A dominance occurs when a particular action or nonaction takes priority over other options within a search space; these have proven to be beneficial in a variety of AI search methods and algorithms (Blake & Gent, 2024; Chu & Stuckey, 2015; Davis et al., 1962). Blake and Gent (2024) describe two types of dominances used for solving solitaire games. The first of these is a move that can be taken in a given situation and will not need to be undone later. Thus, if a solution exists, then a solution exists given the dominance move is performed. A simple example of this dominance is moving an ace or a two to the foundation, actions that are always safe to make. The second type of dominance is one where a possible move is not played in a given situation, since if a solution exists, then it exists without making that move.

Multiple dominances are employed in the implementation of Solvitaire. One of the simpler dominances concerns the safe movement of cards to the foundation. This dominance states that cards can be automatically moved to the foundation if they are at most two more in value than the current card in the foundation of the opposite color and at most three more in value than the current card in the foundation of the same color and other suit (Blake & Gent, 2024). Additionally, worrying back (moving a card from the foundation back to the tableau) can be disallowed if the card would be immediately moved to the foundation by the aforementioned

dominance (Bjarnason et al., 2007; Blake & Gent, 2024). This eliminates potential for a loop where a card is repeatedly worried back to the tableau and played to the foundation in quick succession. Note that these dominances apply only to cards in the tableau, not in the stock, as playing a card in the stock affects the available cards due to the turn 3 structure of standard Klondike solitaire.

When Blake and Gent (2024) studied the code of previous solitaire solvers (Birrell, 2017; Wolter, 2013), they noticed an additional, less obvious, dominance in both solvers. This dominance necessitates that moves of built piles on the tableau are only allowed if either the entire pile is being moved or only part of the pile is being moved, and it is possible to build to foundation the card above the top card in the built pile being moved. Blake and Gent (2024) additionally provide a correctness proof of this dominance.

**Streamliners.** The final technique employed by Blake and Gent (2024) is streamlining, a method which partitions the search space to push results in a direction more likely to be successful (Gomes & Sellmann, 2004). This acts as a less strict dominance in solitaire solvers, where certain moves must be made when they are available, but these moves do not necessarily lead to a solution. Ideally, streamliners have a good chance of producing a solution while greatly reducing the search space (Blake & Gent, 2024). A major downside to this strategy is that streamliners can produce false negatives. To eliminate these false negatives, Blake and Gent recommended use of the ‘smart’ streamliner, which runs at 10% time using streamliners before running at full time without using any streamliners. This method cannot produce a false negative and is shown to reduce time by a mean of 2.5 seconds per game.

Two streamliners are implemented in Solvitaire, both as run-time options to the user. The first of these is to immediately move a card to the foundation when it becomes available. This

greatly decreases the search space and leads to solutions in most games (Blake & Gent, 2024). The second streamliner extends symmetry to encapsulate suits of the same color, treating red cards and black cards as two groups in the transposition table, regardless of suit within each color. While not every game will succeed because of the differences between suits when building to the foundation, this streamliner is still often quite helpful.

### *Solvitaire Evaluation*

While Blake and Gent (2024) evaluated solvability percentages for dozens of games, their findings for standard Klondike solitaire are particularly interesting. They were able to achieve a 95% confidence interval of [81.861%, 82.029%] for the solvability of the game. This was done with an average run-time of 75 CPU seconds per game, a significant improvement over previous academic results that averaged more than half an hour per game. This allowed Blake and Gent to analyze far more games than their predecessors, sampling 1 million random instances of solitaire with a total run-time of 870 CPU days. They used a timeout cutoff of one hour, considering any game that was not solved within one hour to be neither solvable nor unsolvable, but unknown. Unknown games were added to both the upper and lower limits of the confidence interval to produce the most conservative confidence interval possible, operating with the assumption that both every unsolvable game is winnable, and every unsolvable game is unwinnable. They employed a 95% binomial confidence interval using Wilson's method (Agresti & Coull, 1998) for all results.

## Methodology

When a solitaire player opens a new game of solitaire on any solitaire application, they encounter seven face up cards. If a player only has limited time to play and wants to ensure the game they are playing is solvable, they might elect to continue starting new games until they see

seven cards they believe give them a good chance of winning. Alternatively, a player seeking to achieve a fast time for a game of solitaire may wait for a favorable deal before attempting to solve it. Despite the advantages associated with knowing the connection between the initial visible cards on the tableau and the impact they have on solvability, little has been published to investigate this relationship. Thus, an extension to Solvitaire (Blake & Gent, 2024) is introduced to close this gap. Initial game states containing visible cards with user-specified characteristics, such as having 4 aces visible, are generated and evaluated using Solvitaire. This sets up the framework for a full analysis of the effects of the initial face up tableau cards on Klondike solitaire solvability.

**Modifications to Solvitaire**

Multiple modifications to Solvitaire were required for this project and can be found in Appendix B. These changes were optimized for ease of use and sometimes resulted in rigid code, breaking away from the general solver format that works for a wide range of solitaire games. The first issue encountered when attempting to run Solvitaire was difficulty using the command-line options to run the program. This issue arose when the program options description, originating from the boost program options library, was unable to recognize any options attached to a command entered through the command line. This issue could have arisen because of a mixture of older libraries and an outdated C++ version or could have a different source altogether. After spending some time troubleshooting this issue, a decision was made to abandon the flexible command line structure and directly input any parameters into the program code. These parameters include setting the game type to Klondike and the timeout to 1 minute. Additionally, the smart streamliner is utilized in this analysis. While this approach of direct parameter insertion

does not change any results, it does mean that the program must be rebuilt whenever a user wishes to change a parameter, such as what game or set of games is being solved.

### *Use of Folders for Game JSON Input*

While Solvitaire supports the input of multiple JSON files, these must be explicitly declared and typed out. A modification was made to the command line helper code to support the selection of a folder, where all files within the folder would be considered as input files. This is hard-coded if no selections are made with the command line but offers functionality if a user is successful in operating the command line with options. When the user selects an input folder, the path of the folder is passed to the input files parser within the command line helper C++ file. The input files parser iterates through the folder found at the path provided and locates each of its files. These files are added as input files to the input file vector of strings, which is then passed to the solver for analysis.

### *Curation of Result Statistics*

When a solitaire game is entered into Solvitaire to be solved, various statistics are output to the command line output, including whether the game was solved, the time it took to determine the solvability of the game, and information on states searched, dominances used, and other potentially useful statistics. In addition, if the game is solvable, a solution is output to the screen, step by step. The statistics, including the vital solvability result, are not output to any file by default. While it appears that a framework for this process exists within the solvability shell file, a simple output to a csv is implemented for ease of use. This occurs at the end of each game, when statistics would normally be output to the command prompt. While this output still occurs, the same statistics are also sent to a csv file named results.csv. If this file does not exist, the file

is created, and column headers are added for each of the columns. This file can easily be renamed or moved to a different directory before running the program again.

## Creation of Game Instance JSONs

One of the advantages of Solvitaire is its ability to calculate solvability for a solitaire game based on a JSON file describing its initial configuration. Since the focus of this project is on the effect of different visible tableau configurations on solvability, the generation of those JSON configurations is key. This generation is performed using a separate C++ program, found in Appendix B. This program is kept separate from Solvitaire to not add to the complexity of the program and because it does not rely on Solvitaire in any manner. C++ was chosen for its slight performance edge over a language such as Python, though instance generation is not computationally expensive in most cases. Multiple functions are defined to generate game states based on conditions placed upon the visible tableau cards. Additionally, the run-time is calculated for JSON generations.

### *Generation of Random Instances*

The generation of random instances is key to understanding how initial visible tableau configurations affect solvability. Not only does it help to define methods for generating solitaire JSON files, it also provides a baseline against which all results can be compared. Multiple functions are declared to assist with the process of creating random instances of solitaire JSON files, with many of these functions utilized in formulating instances with conditions placed upon the visible cards.

**Generate Deck.** The first function defined generates an ordered deck of 52 cards. This function returns a vector of 52 strings, with each string including a value followed by a suit,

surrounded by quotation marks. The quotation marks are key, as it makes conversion to a JSON format simpler.

***Handling Rank 10.*** In Solvitaire, the rank "10" is the only rank that has two characters rather than a single character. This can cause issues in a JSON creator that relies heavily on indexed string operations. Throughout many of the functions in this program, conditional statements are added to ensure all ranks of 10 are properly handled. An example of this occurs in the generate deck function, where all cards containing 10 are added after the cards with a single character rank, as these ranks can easily be extracted as characters from a string. An alternative method for handling 10 could be to replace it with a placeholder character that does not exist elsewhere in the JSON structure. When the final JSON for a state has been generated, this placeholder can be replaced with "10".

**Shuffle Deck.** A shuffle function shuffles a deck of cards, defined as a vector of strings. The function begins by generating a deck of cards using the generate deck function. This deck is shuffled using the Fisher-Yates shuffle algorithm (Durstenfeld, 1964), which swaps elements such that a uniform distribution of permutations is achieved. The way in which these elements are swapped is determined by random numbers occurring between 1 and the size of the deck of cards. These random numbers are generated by a Mersenne Twister pseudo-random number generator (Matsumoto & Nishimura, 1998), which has a much longer period than many pseudo-random number generators commonly used in C++.

**Random State Generator.** A function is defined that can generate a random initial state of solitaire. A shuffled deck is generated using the shuffle deck function. For each initial state in solitaire, both a tableau and a stock must be generated. In Solvitaire, cards are face up if their suit is uppercase; if the suit of a card is lowercase, then that card is face down. In the tableau, 7 of

the cards must be marked as face up, while the remaining cards are face down. The stock is set to be entirely face down. A string variable is declared to hold the string JSON that will be returned from the function. This begins with some JSON notations present in all states of solitaire, such as the leading brace, and these notational pieces are added throughout as necessary.

The tableau cards are assigned first. A loop iterates through each of the 7 piles, with a nested loop controlling the size of each pile, from left to right as seen in Figure 1. When the iterator from the inner and outer loop has the same value, the associated card is face up. This corresponds to the cards at the bottom of each of the piles shown in Figure 1. Otherwise, the card is face down. Each card is popped from the deck when used. After the tableau cards are added to the state string with proper surrounding notation, the remaining cards in the deck are popped onto the stock and the state is returned.

**Random States to a Folder.** It is often helpful to generate a large number of random states at the same time, rather than a single state. This function allows a user to specify a folder name and a number of states they wish to generate. The folder is created if it does not exist, and a loop iterates a number of times equal to the number of states input by the user. In each iteration, a random state is generated and written to a JSON file named game{i}.json, where i represents the current iteration.

### *Selection of Game Instances*

The selection of which initial tableau configurations to consider is not as obvious as one might expect. An analysis of every possible initial card configuration is not computationally feasible. There are $\frac{52!}{(52-7)!} \approx 6.74 \times 10^{11}$ possible configurations of 7 cards that could be visible at game start, a number approaching 700 billion. Each of these configurations would require

solving hundreds of games to reach a confidence interval with any level of certainty. Thus, alternative methods are used to determine which visible tableaus should be generated. Two of these methods are listed below, but there are countless others that may produce interesting results, including those which account for suit and immediate buildability.

**States with a certain number of one or multiple ranks.** One of the most visually appealing tableaus to be had is one with aces present, since aces can immediately be built to the foundations without fear of negative consequence. To generalize this idea, the situation where any number of a rank or ranks is in the visible tableau is considered.

**States where all visible ranks are unique.** Juxtaposed to the situation where many of a single rank is present is the situation where no ranks are repeated in the visible tableau. The investigation of this situation allows for an interesting comparison between a visible tableau containing no unique ranks and multiple of a single rank, particularly with tableaus that contain four of a rank. It is natural to consider whether it is better for a tableau to contain cards spaced out across multiple ranks or congregated around a single rank.

### *Generation of Instances with a Common Theme*

Both described state situations are implemented in the JSON generator. These implementations contain a similar structure to the implementation for a random state but differ at key points.

**State from a Set Visible Tableau.** This function does the bulk of the work in generating states from visible tableaus with conditions placed upon them. The function accepts an argument of a vector of strings, which represent cards. After a deck is instantiated and shuffled, each card present in the cards argument is removed from the deck. The function continues identically to the random state generator, with one key difference. At any point where a card would be capitalized

to show that it is face up, a card will instead be taken from the cards argument, rather than the deck. This results in a state that contains each of the input cards as a visible card in the tableau in the order they are input.

**Shuffle Cards.** The shuffle cards function works identically to the shuffle deck function, but instead of creating a new deck of cards, it shuffles any number of input cards and returns the shuffled cards. This is used to shuffle cards in the visible tableau before passing them to another function.

**States with a certain number of one rank to a folder.** Unlike the function that takes random states and puts them in a folder, the functions that put conditional states into a folder contain additional logic to determine how these states are generated. For each of the user defined states, a deck is instantiated and shuffled. A loop then progresses through the deck, extracting each card that has a rank matching the user defined rank to a cards vector. When the user input number of cards (between 0 and 4) have been extracted, this loop terminates and a new one begins to populate the rest of the cards vector with the remaining cards at the top of the deck. If the card at the top of the deck is of the specified rank, it is not considered. This process results in a vector of 7 cards representing a visible tableau, which is shuffled and passed to the function that generates a state based on that tableau.

**States with set numbers of two ranks to a folder.** The function to insert set numbers of two ranks into a folder is like the function for a single rank, with a few key differences. The loop that progresses through the deck checks for two ranks, instead of one. When the preordained number of each rank has been extracted to the cards vector, this loop is terminated. This function allows for analysis of tableaus where the value of all seven ranks are known, with only the orientation of the suits left as a mystery.

**States where all visible ranks are unique.** The logic to formulate a state with unique visible ranks is like the logic described above. A key difference is that instead of counting a certain number of a single rank from the deck, cards are popped from the deck to the cards vector so long as their rank does not match the rank of any card already in the cards vector. This occurs until there are 7 cards, at which point the cards are shuffled and a state is produced from the resulting visible tableau.

## Results

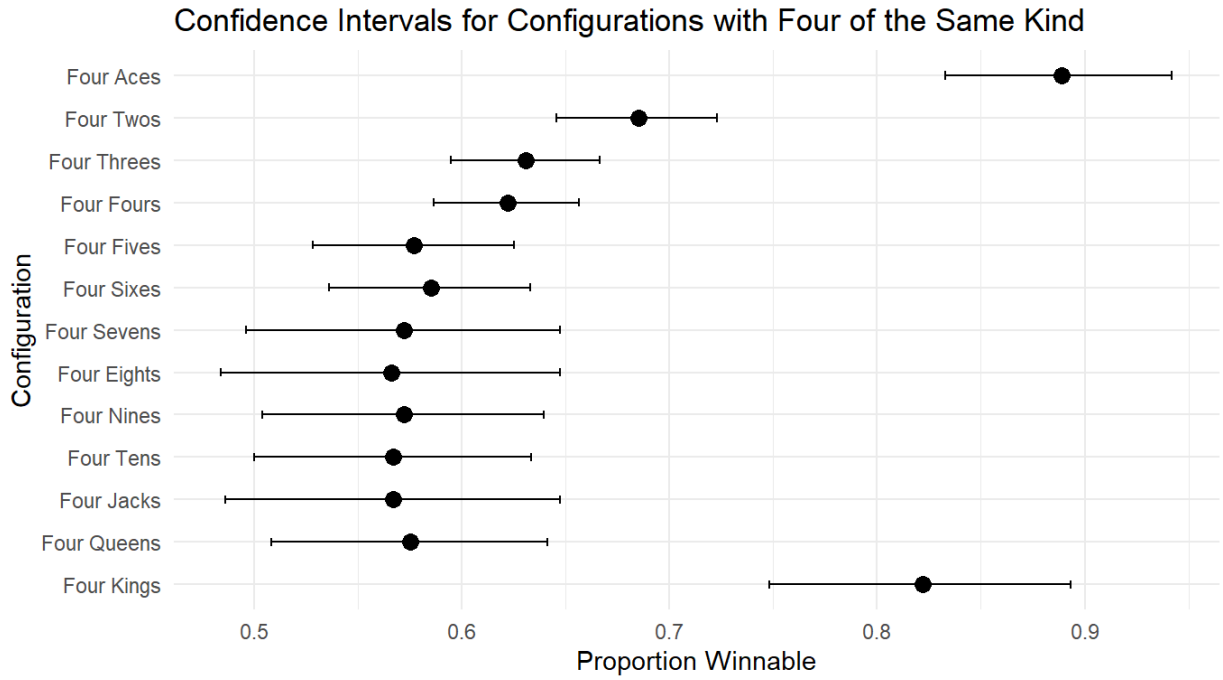
When calculating winnability percentages, a 95% confidence interval using Wilson's Method (Agresti & Coull, 1998) is employed. This is consistent with Blake and Gent (2024), who use this interval in all their results. There is also precedent for the use of a 99% confidence interval (Bjarnason et al., 2007; Bjarnason et al., 2009; Yan et al., 2004), but the 95% confidence interval was chosen to align with the most recent research. Multiple configurations were considered in this analysis, each with a sample of 1000 games. The solvability results of all considered configurations can be found in Appendix A, along with run-times for each configuration.

## Discussion

The case where all four cards of a single rank is present in the tableau is intriguing; a visualization showing the results of each possible configuration of this type is shown in Figure 2. These results present a significant difference between the solvability of games where all four aces or kings are present and games where all four of a different rank are present. This demonstrates the importance of early aces when solving a game, as these cards can always be safely built from the tableau to the foundations. Additionally, kings are valuable as they are the only rank that can be moved to an empty pile.

**Figure 2**

*Confidence Intervals for Configurations with Four of the Same Rank*

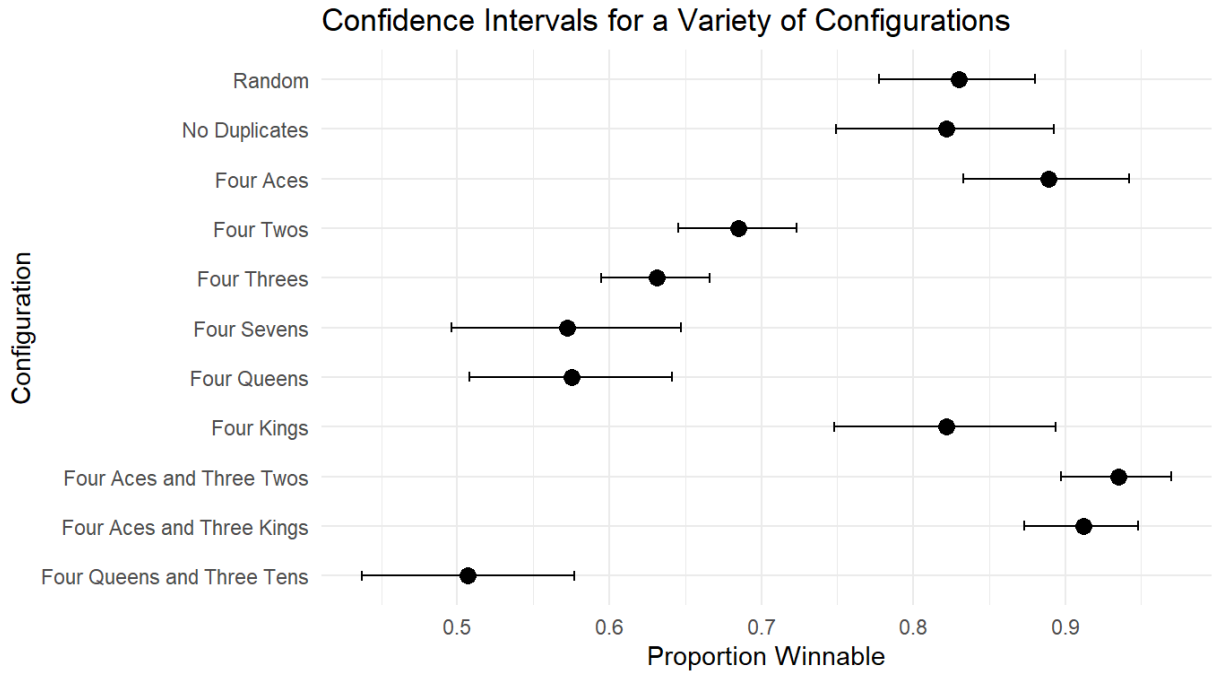



Visible tableaus with four of a rank that is not an ace or a king, however, tend to be solvable less often than random configurations. This suggests that most rank homogeneous configurations of the visible tableau are detrimental to solvability. The high difference in performance between kings and cards such as twos and threes was not expected intuitively, but shows the value of early kings in solitaire gameplay. It is also noteworthy that configurations with four twos, four threes, and four fours had significantly less timeouts than other configurations with four the same rank, resulting in smaller confidence intervals. The magnitude of this difference is found in Table A1 in Appendix A.

A few of the more intriguing four-of-a-kind configurations from Figure 2 were combined with additional configurations to form the visualization found in Figure 3.

**Figure 3**

*Confidence Intervals for Interesting Configurations of Initial Cards*




Some of these configurations include completely random tableaus, which can be considered a control group, visible tableaus where no cards are of a duplicate rank, and three visible tableaus where all seven ranks are predetermined. Two of these represent visible tableaus that are thought of as advantageous to the player, such as four aces and three twos or four aces and three kings. The third example is of four queens and three tens, which may be considered one of the worst possible visible tableaus. These results indicate that four of the same rank is disadvantageous except when those four are kings, which perform similarly to random tableaus, or the four cards are aces, which outperform random tableaus. Additionally, the tableaus with four aces and three twos slightly outperform the tableaus with four aces and three kings, despite four kings significantly outperforming four twos. This appears to suggest that initial buildability, particularly in the foundations, is important for solvability.

One observation evident through an analysis of solvability results is that the confidence intervals shown are much larger than anything obtained by Blake and Gent (2024). This discrepancy in results is in part due to limited run-time and is explained further in the Limitations section. There are also a higher number of unknown results than there were in recent analysis by Blake and Gent, shown in Table A1. This is because of a shorter timeout period set to alleviate some of the run-time burden. These unknown values work together with the small sample size to widen the confidence intervals.

Table 2A contains interesting information concerning the run-times of each of the configurations. These run-times must be contextualized by their maximum time per game of solitaire, which is set at 1 minute. The average time per game at this timeout hovers at around 5 seconds and is heavily influenced by the number of timeouts that occur. In fact, many games can be proven solvable or unsolvable in less than a tenth of a second; games that cannot often take much longer to evaluate. This is an unfortunate result of utilizing an exhaustive search method, but a necessary one to ensure accurate results.

While it is tempting to concede that all games which result in a timeout are unsolvable, it seems likely that the opposite is true. Blake and Gent (2024) reported games in other versions of solitaire to be proven solvable at a search depth exceeding 190 million, while another game was proven unsolvable at a depth over 27 million. These extreme examples illustrate how complex some positions of solitaire are and how one cannot make an assumption regarding the solvability of a game based on it not being solvable within a certain timeframe or search depth.

**Limitations**

This analysis has multiple limitations that help to explain the wide range of confidence intervals and provide insight into possible improvements. The most important limitation with

these results is time. There is little variation in the configurations included in this analysis compared to the possible configurations that can be created using current JSON generator functions. Additionally, JSON generation functions for configurations that consider initial buildability or suits in general can be created to aid in this analysis.

Time also has a sizeable impact on the quality of results from each solvability analysis. Blake and Gent (2024) considered 1 million games when they narrowed their solvability confidence interval to less than a tenth of a percent, while only 1000 games of each configuration are considered in this analysis. Additionally, Blake and Gent allowed for up to one hour before considering a game to be timed out, while this implementation only allows for a minute before timing out. Both deficiencies were caused only by a lack of CPU time (and memory), offering room for significant improvements with increased resources.

**Steps Towards a Better Solver**

This project has been concerned with evaluating the impact of the initial visible tableau cards on a game of thoughtful solitaire, resulting in an upper bound for solvability in the standard game. More than that, the consideration of varying visible tableau configurations brings together the perfect information world of thoughtful solitaire and the partially observable world of standard solitaire. By treating a game of standard solitaire like it is thoughtful solitaire, excellent solvers such as Solvitaire can be utilized to calculate accurate upper bounds to solitaire for a position in any game, not just at the beginning of a game. If this analysis takes place after each move in a game of solitaire, a player can learn how new discoveries of hidden cards from the stock and from the tableau affects the probability of a game being solvable. If that player is playing a version of solitaire where undoing moves is allowed, such as in Windows Solitaire,

then the obtained winnability percentage is no longer an upper bound to winnability, but the actual probability of solving the game.

Despite debates over the exact chances of winning a game of solitaire, researchers agree that not every game is winnable (Bjarnason et al., 2007; Blake & Gent, 2024). Many casual players understand this as well and will restart games when they determine a game unwinnable. Use of a strategy reliant on running a solvability analysis after every move will help inform such a player when a game might be unwinnable. While this method is not practical in the near term due to the computational costs associated with calculating accurate solvability percentages, it is still theoretically a possible strategy for a player interested in how revealed information affects solvability chances in solitaire.

One additional possible application of the blending of thoughtful and traditional solitaire is in the development of a robust, computationally expensive solitaire solver. The design for such a solver begins with considering all possible moves at a given state. A solvability percentage is then calculated based on the results of each of these moves. The move that yields the highest solvability chances is the move selected by the solver, and the process repeats itself until a solution is found or all solvability percentages are equal to zero. While this is far from a practical solver and might not even be an accurate solver, it offers an interesting look into how perfect information solvers can interact with partially observable systems.

## Conclusion

Klondike solitaire is a game that is enjoyed by millions of people daily (Jensen, 2020) and is engrained as a classic time-wasting game in our society, standard on nearly every operating system. Recent advancements in computing technology have allowed for a sophisticated analysis of the solvability of this game in ways unseen before (Blake & Gent,

2024). While many people have certainly been curious about how likely it is that a game of solitaire is solvable, it is probable that these same people have pondered how likely it is that the game they just started is solvable, based on just seven visible cards. This thesis aims to shed some light on this previously unanswered question. The hope is that this research may ignite a conversation around calculating the solvability of solitaire, not only before you deal the cards, but also from an established position in the game.

## References

Agresti, A., & Coull, B. A. (1998). Approximate is better than "exact" for interval estimation of binomial proportions. *The American Statistician*, *52*(2), 119–126. https://doi.org/10.2307/2685469

Akagi, Y., Kishimoto, A., & Fukunaga, A. (2010, August). On transposition tables for single-agent search and planning: Summary of results. In *Proceedings of the International Symposium on Combinatorial Search* (Vol. 1, No. 1, pp. 2-9). https://doi.org/10.1609/socs.v1i1.18164

Birrell, M. (2017). Klondike-Solver. GitHub. https://github.com/ShootMe/Klondike-Solver

Bjarnason, R., Fern, A., & Tadepalli, P. (2009, October). Lower bounding Klondike solitaire with Monte-Carlo planning. In *Proceedings of the International Conference on Automated Planning and Scheduling* (Vol. 19, pp. 26-33). https://doi.org/10.1609/icaps.v19i1.13363

Bjarnason, R., Tadepalli, P., & Fern, A. (2007). Searching solitaire in real time. *ICGA Journal*, *30*(3), 131–142. https://doi.org/10.3233/ICG-2007-30302

Blake, C., & Gent, I. P. (2024). The Winnability of Klondike Solitaire and Many Other Patience Games. *arXiv preprint arXiv:1906.12314*. https://doi.org/10.48550/arXiv.1906.12314

Cavendish. (1890). *Patience games*. Thos. De La Rue & Co.

Chu, G., & Stuckey, P. J. (2015). Dominance breaking constraints. *Constraints*, *20*, 155-182. https://doi.org/10.1007/s10601-014-9173-7

Davis, M., Logemann, G., & Loveland, D. (1962). A machine program for theorem-proving. *Communications of the ACM*, *5*(7), 394-397. https://doi.org/10.1145/368273.368557

Durstenfeld, R. (1964). Algorithm 235: Random permutation. *Communications of the ACM*, *7*(7), 420. https://doi.org/10.1145/364520.364540

Eckhardt, R. (1987). Stan Ulam, John von Neumann, and the Monte Carlo method. *Los Alamos Science*, *15*, 131–141. https://doi.org/10.2172/1054744

Garreau, J. (1994, March 9). Office minefield. *Washington Post*. https://www.washingtonpost.com/archive/lifestyle/1994/03/09/office-minefield/3b74132a-5f0a-455f-a04e-6171d023149b/

Gelly, S., & Silver, D. (2007, June). Combining online and offline knowledge in UCT. In *Proceedings of the 24th international conference on Machine learning* (pp. 273-280). https://doi.org/10.1145/1273496.1273531

Gent, I. P., Petrie, K. E., & Puget, J. F. Chapter 10-symmetry in constraint programming. *Handbook of Constraint Programming*, 329-376. https://doi.org/10.1016/S1574-6526(06)80014-3

Gomes, C., & Sellmann, M. (2004, September). Streamlined constraint reasoning. In *International Conference on Principles and Practice of Constraint Programming* (pp. 274-289). Berlin, Heidelberg: Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-30201-8_22

Greenblatt, R. D., Eastlake III, D. E., & Crocker, S. D. (1967, November). The Greenblatt chess program. In *Proceedings of the November 14-16, 1967, fall joint computer conference* (pp. 801-810). https://doi.org/10.1145/1465611.1465715

Jensen, P. (2020, May 22). *Celebrating 30 Years of Microsoft Solitaire with Those Oh-So-Familiar Bouncing Cards*.

https://web.archive.org/web/20200522210053/https://news.xbox.com/en-us/2020/05/22/celebrating-30-years-microsoft-solitaire/

Matsumoto, M., & Nishimura, T. (1998). Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, *8*(1), 3-30. https://doi.org/10.1145/272991.272995

Morehead, A. H., & Mott-Smith, G. (1949). *The Complete Book of Solitaire and Patience Games*. Grosset & Dunlap.

Partlett, D. (1980). *The Penguin Book of Patience*. Penguin.

Ross, A., & Healey, F. (1963). Patience Napoléon. *Proceedings of the Leeds Philosophical and Literary Society (Literary and Historical Section), 10,* pp. 137–190.

Shannon, C. E. (1950). XXII. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, *41*(314), 256-275. https://doi.org/10.1080/14786445008521796

Smith, B. M. (2005). Caching search states in permutation problems. In *Principles and Practice of Constraint Programming-CP 2005: 11th International Conference, CP 2005, Sitges, Spain, October 1-5, 2005. Proceedings 11* (pp. 637-651). Springer Berlin Heidelberg. https://doi.org/10.1007/11564751_47

Wolter, J. (2013, April 10). *Experimental Analysis of Various Solitaire Games*. Polymorphic Solitaire. https://politaire.com/article/intro.html

Yan, X., Diaconis, P., Rusmevichientong, P., & Roy, B. (2004). Solitaire: Man versus machine. *Advances in Neural Information Processing Systems*, *17*. https://proceedings.neurips.cc/paper_files/paper/2004/hash/48c3ec5c3a93a9e294a8a6392ccedeb4-Abstract.html

## Appendix A

## Solvability and Training Statistics

**Table A1**

*Solvability Statistics of Klondike Solitaire by Starting Configuration*

| Configuration | Sample | Winnable | Unwinnable | Unknown | 95% Confidence Interval |
|---|---|---|---|---|---|
| Random | 1,000 | 830 | 142 | 28 | [0.7775, 0.88] |
| No Duplicates | 1,000 | 822 | 130 | 48 | [0.7491, 0.8925] |
| Four Aces | 1,000 | 889 | 76 | 35 | [0.833, 0.942] |
| Four Twos | 1,000 | 685 | 305 | 10 | [0.6455, 0.723] |
| Four Threes | 1,000 | 631 | 363 | 6 | [0.5946, 0.6664] |
| Four Fours | 1,000 | 622 | 373 | 5 | [0.5865, 0.6565] |
| Four Fives | 1,000 | 577 | 405 | 18 | [0.5281, 0.6253] |
| Four Sixes | 1,000 | 585 | 397 | 18 | [0.5362, 0.6332] |
| Four Sevens | 1,000 | 572 | 383 | 45 | [0.4961, 0.6473] |
| Four Eights | 1,000 | 566 | 383 | 51 | [0.4841, 0.6474] |
| Four Nines | 1,000 | 572 | 391 | 37 | [0.5041, 0.6393] |

*Solvability Statistics of Klondike Solitaire by Starting Configuration*

| Configuration | Sample | Winnable | Unwinnable | Unknown | 95% Confidence Interval |
|---|---|---|---|---|---|
| Four Tens | 1,000 | 567 | 397 | 36 | [0.5001, 0.6334] |
| Four Jacks | 1,000 | 567 | 383 | 50 | [0.4861, 0.6474] |
| Four Queens | 1,000 | 575 | 389 | 36 | [0.5081, 0.6413] |
| Four Kings | 1,000 | 822 | 129 | 49 | [0.7481, 0.8935] |
| Four Aces and Three Twos | 1,000 | 935 | 44 | 21 | [0.897, 0.9697] |
| Four Aces and Three Kings | 1,000 | 912 | 68 | 20 | [0.8728, 0.948] |
| Four Queens and Three Tens | 1,000 | 507 | 454 | 39 | [0.437, 0.5769] |

**Table A2**

*Training Statistics of Klondike Solitaire by Starting Configuration*

| Configuration | Mean Nodes | Average Time (seconds) | Total Time (minutes) |
|---|---|---|---|
| Random | 2,397,449.41 | 3.37 | 56.21 |
| No Duplicates | 2,785,854.49 | 4.03 | 67.11 |
| Four Aces | 2,184,193.39 | 3.23 | 53.77 |
| Four Twos | 992,446.44 | 1.14 | 19.06 |
| Four Threes | 566,841.01 | 1.20 | 19.98 |
| Four Fours | 619,836.24 | 1.07 | 17.82 |
| Four Fives | 1,379,794.54 | 2.00 | 33.33 |
| Four Sixes | 1,464,438.75 | 2.24 | 37.41 |
| Four Sevens | 3,058,531.70 | 4.45 | 74.09 |
| Four Eights | 3,305,252.89 | 5.29 | 88.18 |
| Four Nines | 2,980,984.86 | 4.38 | 72.99 |
| Four Tens | 2,549,982.02 | 4.06 | 67.69 |
| Four Jacks | 3,328,703.97 | 4.63 | 77.10 |

*Training Statistics of Klondike Solitaire by Starting Configuration*

| Configuration | Mean Nodes | Average Time (seconds) | Total Time (minutes) |
|---|---|---|---|
| Four Queens | 2,342,406.04 | 3.49 | 58.14 |
| Four Kings | 2,536,141.88 | 4.13 | 68.90 |
| Four Aces and Three Twos | 1,648,331.11 | 1.90 | 31.66 |
| Four Aces and Three Kings | 1,417,032.00 | 2.37 | 39.57 |
| Four Queens and Three Tens | 2,864,098.85 | 3.71 | 61.76 |

## Appendix B

## Code Availability

The code used in this analysis is available for both the JSON generator and the modified version of Solvitaire. The JSON generator can be found at https://github.com/joshstoneback/JSON-State-Generator and the modified version of Solvitaire at https://github.com/joshstoneback/Modified-Solvitaire.